we add one edge from the root tree of Tree 1 to the root of Tree 0, to merge them. On the right, we show how this change the data structure: now we only have two trees, but the height of tree 0 is now larger than before the merge.

So, now merging two partitions requires a constant-time number of changes, plus the number of look-ups needed to find the two roots. Finding the set to which an element belongs to (or seeing if two elements belong to the same partition) requires logarithmic time on average (remember when we introduced trees?[11]) but linear time in the worst case. That's because, when merging partitions, we might always get unlucky with the choice of which tree's root we set as child of the other. By each time randomly choosing which root is going to be used as a child of the other, we can make this worst-case scenario unlikely... but it would still be possible (although extremely unlucky) to find ourselves in an edge situation like the one depicted in figure 5.9. This means that our worst-case scenario for `merge` still requires `O(n)` look-ups and, what's worst, now even `findPartition` has linear running time.